In [7]:
```
#Name: Sumita Kuswar
#Student ID: 2461844
#Worksheet-7
```

In [8]:
```
# Section: Custom Vs. Scikit Learn Built Decision Tree.
# In this experiment, we will build a decision tree both from scratch (custom implementation) and using
# Scikit-learn, applied to the Iris dataset. We will then compare the accuracy of both models on the test
# set.

# Step -1- Building a Custom Decision Tree with Information Gain:
# Custom Built Decision Tree:

import numpy as np

class CustomDecisionTree:
    def __init__(self, max_depth=None):
        """
        Initializes the decision tree with the specified maximum depth.
        Parameters:
        max_depth (int, optional): The maximum depth of the tree. If None, the tree is expanded until all
        leaves are pure or contain fewer than the minimum samples required to split.
        """
        self.max_depth = max_depth
        self.tree = None

    def fit(self, X, y):
        """
        Trains the decision tree model using the provided training data.
        Parameters:
        X (array-like): Feature matrix (n_samples, n_features) for training the model.
        y (array-like): Target labels (n_samples,) for training the model.
        """
        self.tree = self._build_tree(X, y)

    def _build_tree(self, X, y, depth=0):
        """
        Recursively builds the decision tree by splitting the data based on the best feature and threshold

        Parameters:
        X (array-like): Feature matrix (n_samples, n_features) for splitting.
        y (array-like): Target labels (n_samples,) for splitting.
        depth (int, optional): Current depth of the tree during recursion.
        Returns:
        dict: A dictionary representing the structure of the tree, containing the best feature index,
        threshold, and recursive tree nodes.
        """
        num_samples, num_features = X.shape
        unique_classes = np.unique(y)

        if len(unique_classes) == 1:
            return {'class': unique_classes[0]}
        if num_samples == 0 or (self.max_depth is not None and depth >= self.max_depth):
            return {'class': np.bincount(y).argmax()}

        best_info_gain = -float('inf')
        best_split = None

        for feature_idx in range(num_features):
            thresholds = np.unique(X[:, feature_idx])
            for threshold in thresholds:
                left_mask = X[:, feature_idx] <= threshold
                right_mask = ~left_mask
                left_y = y[left_mask]
                right_y = y[right_mask]
                if len(left_y) == 0 or len(right_y) == 0:
                    continue
                info_gain = self._information_gain(y, left_y, right_y)
                if info_gain > best_info_gain:
                    best_info_gain = info_gain
                    best_split = {
                        'feature_idx': feature_idx,
                        'threshold': threshold,
                        'left_mask': left_mask,
                        'right_mask': right_mask,
                    }

        if best_split is None:
            return {'class': np.bincount(y).argmax()}

        left_tree = self._build_tree(X[best_split['left_mask']], y[best_split['left_mask']], depth + 1)
        right_tree = self._build_tree(X[best_split['right_mask']], y[best_split['right_mask']], depth + 1)

        return {
            'feature_idx': best_split['feature_idx'],
            'threshold': best_split['threshold'],
            'left_tree': left_tree,
            'right_tree': right_tree
        }

    def _information_gain(self, parent, left, right):
        """
        Computes the Information Gain between the parent node and the left/right child nodes.
        Parameters:
        parent (array-like): The labels of the parent node.
        left (array-like): The labels of the left child node.
        right (array-like): The labels of the right child node.
        Returns:
        float: The Information Gain of the split.
        """
        parent_entropy = self._entropy(parent)
        left_entropy = self._entropy(left)
        right_entropy = self._entropy(right)
        weighted_avg_entropy = (len(left) / len(parent)) * left_entropy + (len(right) / len(parent)) * right_entropy
        return parent_entropy - weighted_avg_entropy

    def _entropy(self, y):
        """
        Computes the entropy of a set of labels.
        Parameters:
        y (array-like): The labels for which entropy is calculated.
        Returns:
        float: The entropy of the labels.
        """
        if len(y) == 0:
            return 0.0
        class_counts = np.bincount(y)
        class_probs = class_counts / len(y)
        class_probs = class_probs[class_probs > 0]
        return -np.sum(class_probs * np.log2(class_probs))

    def predict(self, X):
        """
        Predicts the target labels for the given test data based on the trained decision tree.
        Parameters:
        X (array-like): Feature matrix (n_samples, n_features) for prediction.
        Returns:
        list: A list of predicted target labels (n_samples,).
        """
        return [self._predict_single(x, self.tree) for x in X]

    def _predict_single(self, x, tree):
        """
        Recursively predicts the target label for a single sample by traversing the tree.
        Parameters:
        x (array-like): A single feature vector for prediction.
        tree (dict): The current subtree or node to evaluate.
        Returns:
        int: The predicted class label for the sample.
        """
        if 'class' in tree:
            return tree['class']
        feature_val = x[tree['feature_idx']]
        if feature_val <= tree['threshold']:
            return self._predict_single(x, tree['left_tree'])
        else:
            return self._predict_single(x, tree['right_tree'])
```

In [13]:
```
# Step -2- Load and Split the Iris Datasets:
# Load and Split the IRIS Dataset:

import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

In [14]:
```
# Step -3- Train and Evaluate a Custom Decision Tree:
# Train and Evaluate a Custom Decision Tree:

custom_tree = CustomDecisionTree(max_depth=3)
custom_tree.fit(X_train, y_train)

y_pred_custom = custom_tree.predict(X_test)

accuracy_custom = accuracy_score(y_test, y_pred_custom)
print(f"Custom Decision Tree Accuracy: {accuracy_custom:.4f}")
```

```
Custom Decision Tree Accuracy: 1.0000
```

In [15]:
```
# Step -4- Train and Evaluate a Scikit Learn Decision Tree:
# Train and Evaluate a Scikit Learn Decision Tree:

sklearn_tree = DecisionTreeClassifier(max_depth=3, random_state=42)
sklearn_tree.fit(X_train, y_train)

y_pred_sklearn = sklearn_tree.predict(X_test)

accuracy_sklearn = accuracy_score(y_test, y_pred_sklearn)
print(f"Scikit-learn Decision Tree Accuracy: {accuracy_sklearn:.4f}")
```

```
Scikit-learn Decision Tree Accuracy: 1.0000
```

In [16]:
```
# Step -5- Result Comparison:
# Result Comparison:

print("Accuracy Comparison:")
print(f"Custom Decision Tree: {accuracy_custom:.4f}")
print(f"Scikit-learn Decision Tree: {accuracy_sklearn:.4f}")
```

```
Accuracy Comparison:
Custom Decision Tree: 1.0000
Scikit-learn Decision Tree: 1.0000
```

In [ ]:
```
# 3
  # Exercise - Ensemble Methods and Hyperparameter Tuning.
  # Using the Wine Dataset from scikit-learn
  # 1. Implement Classification Models:
  # • Train a Decision Tree Classifier and a Random Forest Classifier using scikit-learn.
  # • Compare the models based on their F1 scores.
  # 2. Hyperparameter Tuning:
  # • Identify three hyperparameters of the Random Forest Classifier.
  # • Perform hyperparameter tuning using GridSearchCV to optimize these parameters.
  # • Take hints from the scikit-learn documentation to guide the implementation.
  # 3. Implement Regression Model:
  # • Train a Decision Tree Regressor and a Random Forest Regressor using scikit-learn.
  # • Identify three parameters for Random Forest Regressio and Perform hyperparameter tuning using
  # RandomSearchCV to optimize these parameters.

  from sklearn.datasets import load_wine
  from sklearn.ensemble import RandomForestClassifier
  from sklearn.metrics import f1_score
  from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
  from sklearn.tree import DecisionTreeRegressor
  from sklearn.ensemble import RandomForestRegressor
  from sklearn.datasets import load_diabetes
  from sklearn.metrics import r2_score
  from scipy.stats import randint

  # Wine classification: Decision Tree and Random Forest

  wine = load_wine()
  X_wine = wine.data
  y_wine = wine.target

  X_train_w, X_test_w, y_train_w, y_test_w = train_test_split(
      X_wine, y_wine, test_size=0.2, random_state=42
  )

  dt_clf = DecisionTreeClassifier(random_state=42)
  dt_clf.fit(X_train_w, y_train_w)
  y_pred_dt = dt_clf.predict(X_test_w)
  f1_dt = f1_score(y_test_w, y_pred_dt, average="macro")
  print(f"Decision Tree F1 (macro): {f1_dt:.4f}")

  rf_clf = RandomForestClassifier(random_state=42)
  rf_clf.fit(X_train_w, y_train_w)
  y_pred_rf = rf_clf.predict(X_test_w)
  f1_rf = f1_score(y_test_w, y_pred_rf, average="macro")
  print(f"Random Forest F1 (macro): {f1_rf:.4f}")

  print("F1 Comparison (Wine):")
  print(f"Decision Tree : {f1_dt:.4f}")
  print(f"Random Forest : {f1_rf:.4f}")

  # Hyperparameter Tuning for Random Forest Classifier with GridSearchCV

  param_grid = {
      "n_estimators": [50, 100, 200],
      "max_depth": [None, 5, 10, 20],
      "max_features": ["sqrt", "log2"],
  }

  grid_search = GridSearchCV(
      estimator=RandomForestClassifier(random_state=42),
      param_grid=param_grid,
      cv=5,
      scoring="f1_macro",
      n_jobs=-1,
      verbose=1
  )

  grid_search.fit(X_train_w, y_train_w)

  print("Best parameters (RF classification):", grid_search.best_params_)
  print("Best CV F1 score:", grid_search.best_score_)

  best_rf_clf = grid_search.best_estimator_
  y_pred_best_rf = best_rf_clf.predict(X_test_w)
  f1_best_rf = f1_score(y_test_w, y_pred_best_rf, average="macro")
  print(f"Test F1 with best RF: {f1_best_rf:.4f}")

  # Regression: Decision Tree Regressor and Random Forest Regressor

  reg_data = load_diabetes()
  X_reg = reg_data.data
  y_reg = reg_data.target

  X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
      X_reg, y_reg, test_size=0.2, random_state=42
  )

  dt_reg = DecisionTreeRegressor(random_state=42)
  dt_reg.fit(X_train_r, y_train_r)
  y_pred_dt_reg = dt_reg.predict(X_test_r)
  r2_dt = r2_score(y_test_r, y_pred_dt_reg)
  print(f"Decision Tree Regressor R2: {r2_dt:.4f}")

  rf_reg = RandomForestRegressor(random_state=42)
  rf_reg.fit(X_train_r, y_train_r)
  y_pred_rf_reg = rf_reg.predict(X_test_r)
  r2_rf = r2_score(y_test_r, y_pred_rf_reg)
  print(f"Random Forest Regressor R2: {r2_rf:.4f}")

  # Hyperparameter tuning for Random Forest Regressor with RandomizedSearchCV

  param_dist = {
      "n_estimators": randint(50, 300),
      "max_depth": [None, 5, 10, 20],
      "max_features": ["auto", "sqrt", "log2"],
  }

  random_search = RandomizedSearchCV(
      estimator=RandomForestRegressor(random_state=42),
      param_distributions=param_dist,
      n_iter=20,
      cv=5,
      scoring="r2",
      random_state=42,
      n_jobs=-1,
      verbose=1
  )

  random_search.fit(X_train_r, y_train_r)

  print("Best parameters (RF regressor):", random_search.best_params_)
  print("Best CV R2:", random_search.best_score_)

  best_rf_reg = random_search.best_estimator_
  y_pred_best_rf_reg = best_rf_reg.predict(X_test_r)
  r2_best_rf = r2_score(y_test_r, y_pred_best_rf_reg)
  print(f"Test R2 with best RF Regressor: {r2_best_rf:.4f}")
```

```
Decision Tree F1 (macro): 0.9425
Random Forest F1 (macro): 1.0000
```

F1 Comparison (Wine):
Decision Tree : 0.9425
Random Forest : 1.0000
Fitting 5 folds for each of 24 candidates, totalling 120 fits